Politecnico di Torino

Ingegneria Informatica (Computer Engineering) LM-32(DM270)

A.a. 2025/2026

Graduation Session March 2026

Managing large hordes of NPCs in Unreal Engine 5

An analysis of Mass ECS system

Supervisors:

Francesco Strada

Corrado Raffaelli

Candidate:

Martina Plumari

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures

Chapter 1

Introduction

1.1 Goal

The primary objective of this thesis is to design and implement a highly scalable NPC simulation using the Unreal Engine Mass Framework, targeting a performance threshold of thousands of concurrent entities on mid-to-high-range consumer hardware. To achieve this, the study focuses on developing a robust architectural pipeline that moves beyond passive background elements, ensuring that agents remain interactable and visually convincing through dynamic animation state management. A core technical challenge involves the implementation of a hybrid navigation system capable of reconciling global path-following with local, reactive steering for moving targets across non-planar terrain. Furthermore, the project serves as a critical feasibility study to evaluate the production readiness of Mass's ECS-based ecosystem. By identifying the current limitations in out-of-the-box avoidance, documentation accessibility, and the technical barrier for non-programming disciplines, this thesis aims to provide a definitive recommendation for the framework's adoption within a professional game development pipeline.

1.2 Target Platform Definition

The target used for performance considerations is a laptop with the following hardware specifications:

- CPU: 12th Generation Intel Core i7-12700H
- RAM: 16GB DDR5-4800 MHz
- GPU: NVIDIA GeForce RTX 3050 Ti (4 GB dedicated memory)

1.3 Thesis Structure

Chapter 2

Entity Component System

The Entity Component system (ECS) is a software architectural pattern specifically engineered to facilitate high-performance simulations. Unlike traditional game development frameworks that rely on deep object hierarchies, ECS is built upon the Data-Oriented Design (DOD) paradigm.

At its core, ECS shifts the focus from "what an object is" to "what data it possesses". By decoupling data from behavior, it enables the construction of complex simulations using granular, reusable modules rather than monolithic, rigid classes.

This chapter first analyses the Entity–Component–System architectural pattern and the reasons it can offer performance advantages over the traditional object-oriented paradigm. It then examins the trade-offs of this type of design compared to the object-oriented one.

2.1 Object-Oriented vs Data-Oriented Design

To understand why ECS has become the industry standard for mass-scaled simulations, we must compare it to the traditional Object-Oriented Design (OOD).

In OOD, functionality is typically extended through inheritance, making the resulting objects inextricably bound to their ancestors through a hard-coded relationship. As a project grows, this can lead to deep and rigid class hierarchies. For instance, when a Soldier class inherits from a Human class, it does not merely get access to human traits, but becomes physically coupled to the entire memory footprint and logic chain of the parent. On the other hand, DOD favors composition; in this case, rather than defining what an entity is (a Soldier), we define what it has (Transform, Health, Ammo). This approach allows us to avoid being forced to choose between two unrelated classes and enables architectural flexibility since the entity is simply a dynamic collection of data components assigned at runtime.

The primary driver for DOD is data locality. Modern CPU speeds vastly outpace memory access times; consequently, performance is often throttled by "memory latency", the delaay incurred when fetching data from RAM. In Object-Oriented Design, objects are frequently scattered throughout memory. Accessing them requires "pointer-chasing" through class hierarchies, leading to frequent cache misses and significant performance penalties in contemporary CPU architectures. On the other hand, in Data-Oriented Design components of an identical type are stored contiguously in memory. When a system processes these components the CPU can accurately predict the next data address and pre-load it into the cache. This maximizes cache coherency, facilitating rapid, linear iteration over thousands of entities.

2.2 The Three Pillars of ECS

As the name suggests, the ECS pattern consists of three main functional layers:

- Entity: it is a unique identifier serving as a handle that associates a set of components. It contains no intrinsic data or behavior. Its sole purpose is to provide a unique index to which stateful data can be mapped;
- Component: granular data container that encapsulate state but contain no logic. As "plain old data" structures, components are stored in dense arrays to optimize memory throughput;
- System: it represents the logic layer of the architecture. Systems are independent units that operate over filtered collections of components by means of declarative queries. Because systems operate on dense arrays with explicit data dependencies, they are inherently suited for multi-threaded execution, allowing the engine to distribute workloads across CPU cores with minimal synchronization overhead.

2.2.1 Trade-offs Analysis

While ECS provides the performance requisite for systems like UE Mass, it introduces specific architectural trade-offs:

- Architectural Complexity: initial planning requires a more rigorous design of component interfaces and system boundaries compared to the more intuitive Object based approach of OOD;
- Decoupling Overhead: the radical separation of data can lead to "component bloat" if the granularity is not managed carefully, potentially resulting in repetitive or trivial data structures.

- Mentality Shift: transitioning from an object-centric logic to Data-Oriented paradigm requires a fundamental shift in how developers conceptualize program flow and state management. This shift takes time, and the steep learning curve may not be strategically feasible for all production timelines or team structures.

Chapter 3

Unreal Engine Mass

While the Entity Component System is a theoretical pattern, Mass is Unreal Engine's specific framework for data-oriented calculations and high-performance simulations. It does not replace the traditional Actor-based system; instead, it provides a parallel infrastructure capable of processing a large number of entities with minimal CPU overhead.

In this chapter we are going to explore the declination of the ECS pattern in the Mass System via the core plugin MassEntity and the additional features available through the addition of standalone plugins offerd by Epic Games.

3.1 MassEntity

At the heart of Mass system lies MassEntity, the core module responsible for the framework's low-level heavy lifting. It manages the execution pipeline, entity lifecycle and the specialized memory layouts required for high-performance computing. Initially introduced as a plugin, MassEntity was integrated into the core engine as of UE5.5, with the standalone plugin version now deprecated.

Within MassEntity, we find specialized implementation of the three pillars of the ECS pattern:

- Entities: Unique identifiers that represent individual simulated objects
- Fragments: Logic-less data structures that serve the role of Components, storing the entity's data
- Processors: Highly optimized logic units that act as Systems, operating on specific sets of Fragments.

In the following sections, we will present an overview of the software architecture of Mass Entity, focusing on data structures, memoy organization and logic execution to illustrate how these core ECS concepts are implemented within framework. Following this high-level overview, we will dive deeper in the analysis of each element, providing practical code examples to demonstrate their implementation and interaction.

3.1.1 Entities

In the Mass framework, an Entity is a lightweight, unique identifier rather than a data-carrying object. It serves as a hook to associate various fragments within the Mass database.

When an entity is created, the `MassEntitySubsystem` assigns it an index and a serial number used to prevent dangling references if the entity is destroyed. This abstraction allows the engine to treat entities as transient, high-performance entries in a table.

3.1.2 Memory Organization: Archetypes and Chunks

In Mass Framework, an Archetype is the logical grouping of entities sharing an identical composition of Fragments and Tags (`FMassArchetypeData`). As entities dynamically gain or lose fragments at runtime, they are migrated between Archetypes. This structural consistency allows the engine to bypass the pointer-chasing common in Object-Oriented Programming (OOP) in favor of linear memory access.

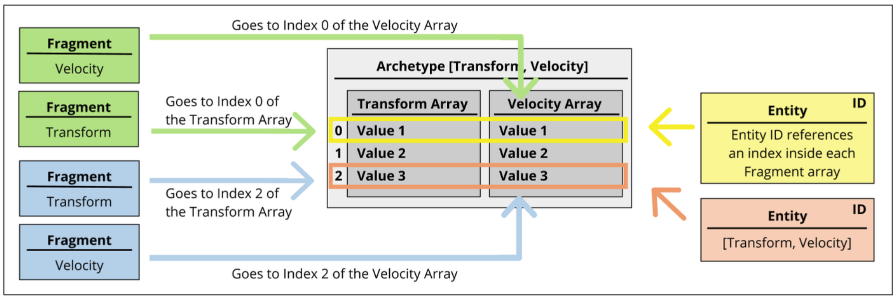

Figure 3.1: Logical grouping of entities into Archetypes based on fragment composition.

To optimize for CPU cache coherency, Mass organizes Archetype data into Chunks (`FMassArchetypeChunk`). A Chunk is a fixed-size memory block that serves as the fundamental unit of both memory allocation and parallel execution. Each Archetype holds an array of Chunks with fragment data and each chunk contains fragments referring to a subset of entities belonging to the archetype.

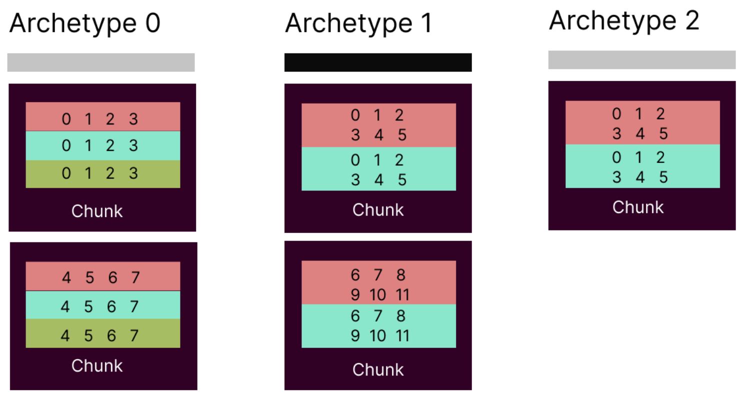

Figure 3.2: Logical organization of Archetype-related data in chunks. Different colors of rows correspond to different fragments while the numbers refer to the Entity ID

To facilitate rapid filtering and query matching, the archetype structure incorporates a bitset representation of its composition. Each bit in this mask corresponds to a specific fragment or tag type. If a bit is set, the archetype contains that component. This bitmasking approach allows the `MassEntitySubsystem` to perform constant-time validation when determines if an archetype satisfies the requirements of a processor's `FMassEntityQuery`.

The default size of the memory block is defined in UMassEntitySettings and it is equal to 128 KB. This size has been established based on current generation caches and could change in future versions of the engine.

Rather than storing all fragments of a single type for an entire Archetype in one massive array, Mass partitions the Archetypes into these 128 KB blocks. Within each chunk, data is organized as struct of arrays, assigning a block of memory to each type of Fragment and Tag instances.

By sizing chunks to fit within modern L2 or L3 caches, Mass ensures that when a processor requests multiple fragments for an entity, the entire row of data is likely already in the cache.

Furthermore, Chunks facilitate safe multi-threading. the MassEntitySubsystem can distribute individual chunks across different CPU cores via the Task Graph, ensuring that no two threads attempt to write to the same memory block simultaneously.

Because the ChunkMemorySize is a fixed constant, the entity density is inversely proportional to the total size of the fragments. Having slim entities means that we can fit more entities in the chunk, improving cache efficiency by reducing the management overhead. When designing fragments, developers should aim for

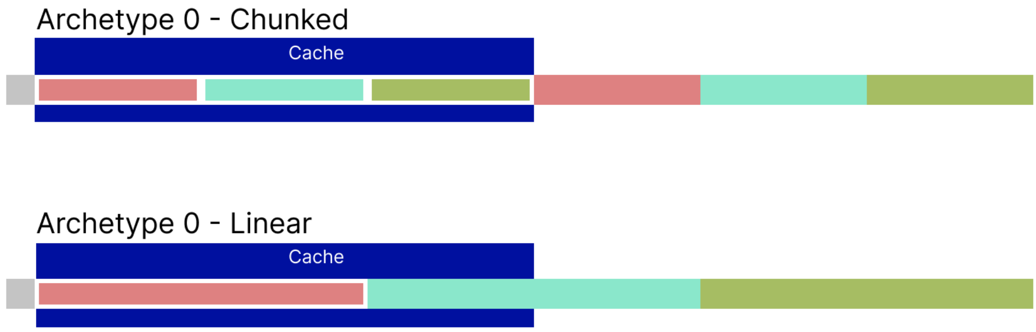

Figure 3.3: The image shows why it is convenient to have chunks sized to fit the cache rather than a single array structure for the whole Archetype.

granularity. Large monolithic fragments reduce the number of entities per chunk, potentially dropping the density to a point where the overhead of chunk-level iteration negates the performance gains of the ECS architecture.

3.1.3 Mass Execution Environment

To coordinate the complex relationship between archetypes, chunks, and entities, Mass utilizes a centralized management layer consisting of the `UMassEntitySubsystem` and the `FMassEntityManager`.

The `FMassEntityManager` acts as the low-level data authority. It maintains the global registry of all active archetypes and provides the logic for structural changes, such as adding or removing fragments and archetype migration. By centralizing these operations, the manager ensures that memory remains contiguous and that entity handles (`FMassEntityHandle`) remain valid throughout their lifecycle.

Surrounding the manager is the `UMassEntitySubsystem`, a `UWorldSubsystem`[1]. that provides a stable API for the rest of the engine. It facilitates the baking of `FMassEntityTemplates` and manages the lifecycle of the simulation relative to the `UWorld`. Together, these components serve as the backbone of the framework, translating high-level commands into the low-level memory operations that define the Mass Framework's performance characteristics.

[1]A World Subsystem in Unreal Engine is a singleton-like class that is automatically managed by the engine and tied directly to the lifecycle of a `UWorld` instance.

3.1.4 Entity Processing and Interaction

To maintain the integrity of the linear memory chunks during multi-threaded execution, Mass employs a deferred-mutation pattern via the Mass Command Buffer. Structural changes, such as spawning entities, destroying them, or altering their archetype through the addition or removal of fragments, are not executed immediately by processors. Instead, these requests are queued as Commands. At designated synchronization points in the frame, the `FMassEntityManager` flushes these buffers, performing the memory-intensive migrations in a batch. This prevents race conditions and ensures that pointers to fragment data remains valid throughout the duration of a processing task.

While processors typically operate on entire batches of entities, individual entity interaction is facilitated through the Entity View (`FMassEntityView`). Because fragments are stored in raw memory buffers, they cannot be accessed through standard pointer dereferencing from an entity handle. The Entity View acts as a transient window into a specific entity's data, resolving the `FMassEntityHandle` into a structured set of pointers for its constituent fragments. This provides a type-safe interface for logic that must operate outside of the standard batch-processing pipeline without sacrificing the performance benefits of the underlying ECS architecture.

3.1.5 Fragments and Tags

Fragments constitute the atomic data units of the Mass Framework. Unlike traditional class properties, fragments are implemented as lightweight `USTRUCTs` inheriting from `FMassFragment` and contain no logic, serving strictly as data containers. By decoupling data from behavior, Mass allows for highly flexible entity composition and optimal memory alignment.

While generic fragments are unique to each entity, the framework provides specialized fragment types to optimize memory footprint and group-level data management.

Entity Fragments

The standard fragment type stored in rows within a chunk. Each entity has its own instance of this data (e.g. `FTransformFragment`), allowing for unique per-entity state.

```

USTRUCT()
struct FSimpleMovementFragment : public FMassFragment
{
  GENERATED_BODY()
```

```
    FVector InitialTarget;
    float Velocity;
};
```

The design of individual fragments should strictly adhere to the principle of locality, where data is grouped according to its specific access pattern or functional aspect. For instance, a `FSimpleMovementFragment`, such as the one in the code example above, should ideally only contain variables relevant to spatial translation, such as `InitialTarget` and `Velocity`.

Integrating unrelated variables into a single monolithic fragment introduces several architectural inefficiencies:

1. **Cache Line Pollution:** CPUs fetch data from memory in fixed-size lines. If a movement processor iterates through a fragment that contains many variables to access velocity data, the CPU is forced to load the adjacent, unrelated variables into the cache. This results in polluted cache lines where a significant portion of the high-speed memory is occupied by data that will not be necessary for execution, effectively wasting memory bandwidth.

2. **Decreased Entity Density:** Because `ChunkMemorySize` is a fixed constant, the number of entities that can coexist within a single chunk is inversely proportional to the total size of its fragments. Increasing the memory footprint of a fragment directly reduces entity density, necessitating more frequent chunk-switching and increasing the management overhead per entity.

3. **Parallelization Constraints:** The Mass Framework achieves high performance through concurrent execution, where different processors can operate on different fragments of the same entity simultaneously. If disparate variables, such as `Health` and `Velocity`, are defined in a single fragment, a processor modifying health will effectively lock the entire fragment. This creates a data dependency that prevents a movement processor from accessing the velocity data in parallel, forcing sequential execution or requiring complex synchronization overhead that negates the benefits of the ECS architecture.

4. **Structural Change Overhead:** Entities in Mass are dynamic; adding or removing fragments triggers a migration to a new Archetype. This process involves a memory copy of the entity's existing fragments. Monolithic fragments significantly increase the volume of data that must be shuffled during these structural changes, even if the modification only involves a single, small variable.

Chunk Fragments

These fragments (`FMassChunkFragment`) are stored once per chunk rather than once per entity. They are ideal for data that is identical for every entity within a specific memory block, such as Level-of-Detail (LOD) information or spatial bounds. By updating a single chunk fragment, a processor can effectively update the state of hundreds of entities simultaneously.

Shared Fragments

Shared fragments (`FMassSharedFragment`) represent data that is common across multiple chunks or even multiple archetypes. They are stored in a central registry and referenced via a handle, making them highly efficient for constant data, such as a shared mesh reference or a global movement configuration, that rarely changes. Here is an example of shared fragment that contains the Player's position:

```

USTRUCT(BlueprintType)
struct FCurrentPlayerPositionFragment : public
    FMassSharedFragment
{
     GENERATED_BODY()

     UPROPERTY(EditAnywhere, BlueprintReadWrite)
     FVector PlayerPosition;
};
```

Tags

Implemented via `FMassTag`, tags are zero-sized fragments that carry no data, having no member properties. They exist solely as bits within the archetype's `FMassTagBitSet`. They are used for high-speed categorization and filtering, allowing processors to include or exclude entities based on boolean-like markers without increasing memory consumption.

To define a custom tag, a developer simply inherits from `FMassTag`, ensuring the struct remains empty:

```

USTRUCT()
struct FDeadTag : public FMassTag
{
     GENERATED_BODY()
};
```

3.1.6 Traits

While fragments and tags provide the low-level data structures for the ECS, traits serve as the high-level configuration layer that bridges the gap between the Unreal Editor and the ECS backend. A trait acts as a high-level recipe that specifies which fragments, tags and shared data should be attached to an entity. This abstraction allows designers to compose complex agents behavior, such as crowd movement, avoidance or visualization, without directly interacting with the underlying implementation.

A trait is implemented in C++ by inheriting from `UMassEntityTrait` and overriding the `BuildTemplate` function:

```

UCLASS(meta = (DisplayName = "NPC Horde Movement"))
class NPCHORDE_API UNPCMovementTrait : public UMassEntityTraitBase
{
    GENERATED_BODY()

protected:
    virtual void BuildTemplate(FMassEntityTemplateBuildContext& BuildContext, const UWorld& World) const override
    {
        FMassEntityManager& EntityManager =
                UE::Mass::Utils::GetEntityManagerChecked(World);

        // Ensuring another trait provides this fragment
        BuildContext.RequireFragment<FTransformFragment>();

        // Adding a Fragment
        BuildContext.AddFragment<FSimpleMovementFragment>();

        // Adding a Tag
        BuildContext.AddTag<FMovingTag>();

        // Creating and adding a shared fragment
        const FConstSharedStruct RandomMovementFragment = EntityManager.GetOrCreateConstSharedFragment(RandomMovement);
        BuildContext.AddConstSharedFragment(RandomMovementFragment);
```

```
    }

    UPROPERTY(Category = "Movement", EditAnywhere, meta = (
   EditInline))
    FMassMovementParameters RandomMovement;
};
```

When the MassEntitySubsystem processes a trait, it executes the BuildTemplate function. During this phase, the trait contributes with its components to the entity template adding necessary fragments and tags to the archetype definition.

Following this construction phase, traits can override ValidateTemplate to provide custom verification logic. ValidateTemplate is a function invoked for all traits once the template composition is complete. It allows native traits to audit the final BuildContext, log configuration errors, or perform last-minute adjustments to ensure structural integrity. By separating construction from validation, the framework ensures that any missing dependencies or invalid parameters are caught in the Editor before the simulation begins, rather than causing runtime failures in the high-performance memory chunks. Here follows an example of template validation function from the MassLODTrait.h file:

```
    bool UMassLODCollectorTrait::ValidateTemplate(const
   FMassEntityTemplateBuildContext& BuildContext, const UWorld&
    World, FAdditionalTraitRequirements& OutTraitRequirements)
   const
{
  // if bTestCollectorProcessor is set to true, we require
   MassLODCollectorProcessor to be enabled.
  if (bTestCollectorProcessor && !ValidateRequiredProcessor(
   this, UMassLODCollectorProcessor::StaticClass()))
  {
    return false;
  }

  return Super::ValidateTemplate(BuildContext, World,
   OutTraitRequirements);
}
```

Traits are primarily utilized during the construction of an FMassEntityTemplate, which is authored via a UMassEntityConfigAsset. This specialized Data Asset acts as a high-level container where designers can aggregate various traits to define an entity's composition and behavior. They often contain parameters that can be edited in the editor tab which are then used to populate the initial values of the fragments.

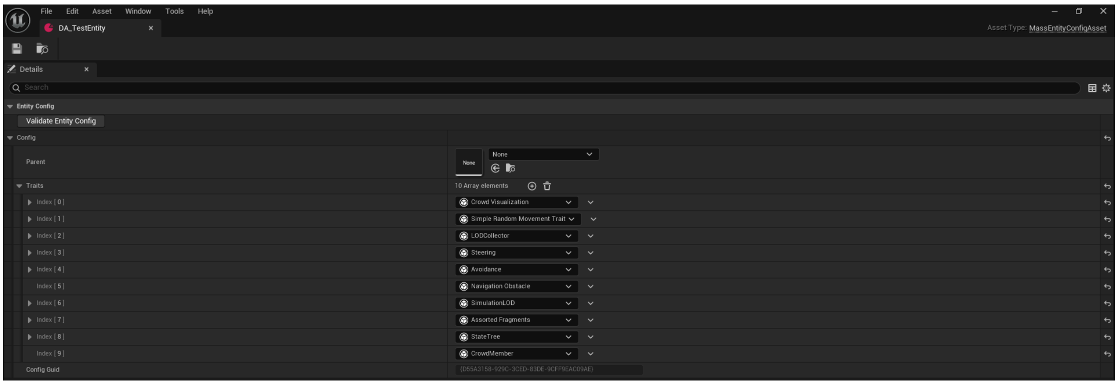

Figure 3.4: Example of Mass Entity Config Asset defining an Entity via editor. It is possible to create one by clicking on the "Add" button in the content drawer, selecting Data Asset and then choosing Mass Entity Config Asset.

While custom C++ traits offer the most control, the Mass Framework provides versatile built-in traits to streamline the development workflow. A notable example is the Assorted Fragments Trait (`UMassAssortedFragmentsTrait`). This trait utilizes an array of `FInstancedStructs`, allowing designers to add fragments directly within the Unreal Editor.

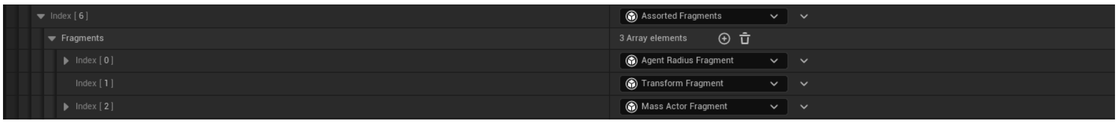

Figure 3.5: Example of Assorted Fragment trait use.

Unlike standard Actor Components, traits do not exist at runtime once the entity has been spawned but are Editor-time construction tools. Once the simulation begins, the trait's responsibility ends, and the entity exists purely as a collection of fragments and tags. This ensures that the high-level flexibility of the Unreal Editor does not compromise the efficiency of the simulation.

3.1.7 Processors and Queries

The System element of the ECS pattern is represented in Unreal Engine by Processors.

Processors are stateless classes (inheriting from `UMassProcessor`) that house the simulation logic. They are designed to be data-agnostic, meaning they do not operate on individual entities through isolated function calls, but rather on contiguous fragments provided by the Entity Manager. By utilizing one or more user-defined Queries, processors filter the global entity population into optimized batches, allowing for high-throughput execution across matching memory chunks.

By default, any class deriving from `UMassProcessor` is automatically registered with the Mass Framework. To ensure synchronization with the rest of the Unreal Engine ecosystem (such as Physics, Animation, and Networking), the Mass Framework organizes processors into Processing Phases. Each phase is mapped to a specific `ETickingGroup`, which dictates when the Mass logic will execute relative to the rest of the engine's frame logic.

By default, processors are added to the `PrePhysics` phase, but developers can redistribute them to optimize parallelization or satisfy data dependencies.

Table 3.1: Unreal Engine 5 Mass Processing Phases and Tick Groups

Processing Phase	**Unreal Tick Group**	**Description**
PrePhysics	`TG_PrePhysics`	The default phase. Used for logic that sets up movement or state before the physics engine runs.
StartPhysics	`TG_StartPhysics`	A specialized phase that marks the beginning of the physics simulation.
DuringPhysics	`TG_DuringPhysics`	Logic that runs concurrently with the physics simulation, ideal for calculations that don't depend on current-frame physics results.
EndPhysics	`TG_EndPhysics`	A phase used to finalize physics-related calculations.
PostPhysics	`TG_PostPhysics`	Used for logic that depends on physics results, such as updating transforms based on collision or rigid body movement.
FrameEnd	`TG_LastDemotable`	A catch-all phase for cleanup or low-priority logic that can be deferred to the end of the frame.

In their constructor, processors define the execution topology by specifying their processing phase and establishing relative execution rules (e.g., `ExecuteBefore` or `ExecuteAfter`). Furthermore, they utilize **Execution Flags** to restrict logic to specific environment types, such as dedicated servers, local clients, or standalone instances.

```
UMyProcessor::UMyProcessor() :
    MyQuery(*this) // Link the query to this processor
{
    // Enables automatic discovery and registration by the MassEntitySubsystem
    bAutoRegisterWithProcessingPhases = true;
    // Assigns execution to a specific engine tick group
    ProcessingPhase = EMassProcessingPhase::PrePhysics;
    // Organizes the processor within a specific functional simulation block
    ExecutionOrder.ExecuteInGroup = UE::Mass::ProcessorGroupNames::Movement;
    // Establishes an execution dependency on another processor
    ExecutionOrder.ExecuteAfter.Add(TEXT("MyMovementProcessor"));
    // Limits execution to specific network roles (Client or Standalone)
    ExecutionFlags = (int32)(EProcessorExecutionFlags::Client | EProcessorExecutionFlags::Standalone);
    // Disables multithreading to allow safe access to Game Thread-only APIs
    bRequiresGameThreadExecution = true;
}
```

While the Mass Framework executes processors in parallel across multiple worker threads by default, they can be constrained to a single-threaded execution on the Game Thread by setting `bRequiresGameThreadExecution` to `true`. This configuration is essential for tasks that must interact with non-thread-safe Unreal Engine systems, such as spawning Actors or modifying UI components, which require a stable execution environment on the main thread.

Users can aggregate processors into logical blocks called Processor Groups. Groups allow for a modular organization of the simulation logic. For example, a Movement group might contain separate processors for path following, steering, and avoidance. By assigning a processor to a group via `ExecuteInGroup`, developers can manage high-level simulation goals rather than individual class dependencies. The exact sequence of logic across these groups is enforced through the `ExecuteBefore` and `ExecuteAfter` constraints. These rules allow the framework to build a dependency graph during initialization. A Visualization Processor should always be set to `ExecuteAfter` the movement processor, to ensure that entities are rendered at their updated location.

Even if it may appear that a processor iterates over every entity simultaneously, the execution is actually performed in batches based on the memory organization of Chunks and Archetypes we previously analyzed. The standard execution flow follows these steps:

1. **Configuration:** in the `ConfigureQueries` function the processor defines its data requirements using one or more query.

2. **Filtering:** the query matches the requirements against the existing archetypes. Entire chunks are skipped if they do not meet the criteria.

3. **Iteration:** the processor calls `ForEachEntityChunk`. For every matching chunk, a lambda function is executed.

4. **Contextual Access:** within the lambda, the processor uses the Mass Execution Context to access fragment views (arrays of data) and perform calculations.

Mass Entity Queries

The `FMassEntityQuery` is the mechanism used to filter and fetch data based on the presence or absence of fragments and tags. Queries are highly optimized to perform bitwise comparisons against archetype bitsets, ensuring that the processor only touches relevant memory.

To maximize parallelization and cache efficiency, queries define precisely how they intend to interact with data. These rules are categorized into Access and Presence requirements. Access requirements define the lock type on the data:

- **ReadOnly:** the processor will only read the data, allowing other read-only processors to access it simultaneously.

- **ReadWrite:** the processor intends to modify the data, requiring exclusive access to prevent race conditions.

Presence requirements define the filtering rules:

- **All:** it mandates that an entity possess every specified fragment and tag in the query. It acts as a strict logical `AND` filter. This is the default presence requirement.

- **Any:** it mandates that an entity posses at least one of the specified fragment and tags. It acts as a logical `OR` filter.

- **None:** it excludes archetypes that contain these components. This is ideal for state changes, for example a Movement processor that needs to ignore entities with a `DeadTag`.
- **Optional:** it accesses the data if it exists, but does not filter out the entity if it is missing.

To apply these filters and access rules, processors must override the `ConfigureQueries` function. This is where the `FMassEntityQuery` is structured before the execution phase begins.

```
void UCheckPlayerProximityProcessor::ConfigureQueries(const TSharedRef<FMassEntityManager>& EntityManager)
{
  EntityQuery.AddRequirement<FTransformFragment>(EMassFragmentAccess::ReadWrite);
  EntityQuery.AddRequirement<FMassMoveTargetFragment>(EMassFragmentAccess::ReadWrite);
  EntityQuery.AddRequirement<FNPCEntityState>(EMassFragmentAccess::ReadWrite);
  EntityQuery.AddRequirement<FMassRepresentationLODFragment>(EMassFragmentAccess::ReadOnly);
  EntityQuery.AddSharedRequirement<FCurrentPlayerPositionFragment>(EMassFragmentAccess::ReadWrite);
  EntityQuery.AddTagRequirement<FDeadTag>(EMassFragmentPresence::None);

    //EntityQuery.RegisterWithProcessor(*this);
}
```

Queries must be registered with the processor to be tracked by the execution pipeline. While the most common pattern is to initialize the `FMassEntityQuery` in the processor's constructor, passing `*this` as the owner, it is also possible to register it explicitly within the `ConfigureQueries` function. The syntax to use is the one commented in the last line of the previous code example.

To process the filtered entities, the `Execute` function calls the query's `ForEachEntityChunk` method. This function takes the execution context and a lambda expression, which defines the logic to be applied to each chunk matching the query's requirements.

```

void MyProcessor::Execute(FMassEntityManager& EntityManager, FMassExecutionContext& Context)
{
```

```
    EntityQuery.ForEachEntityChunk(Context,
                  [this](FMassExecutionContext& Context)
    {
            // Obtain view for the fragments
        TArrayView<FTransformFragment> TransformList = Context.GetMutableFragmentView<FTransformFragment>();
        TArrayView<FSimpleMovementFragment> SimpleMovementList = Context.GetMutableFragmentView<FSimpleMovementFragment>();

            // Obtain the shared parameters from the shared fragment
        const FSimpleRandomMovementParameters MovementParams = Context.GetConstSharedFragment<FSimpleRandomMovementParameters>();

        for (int32 EntityIndex = 0; EntityIndex < Context.GetNumEntities(); EntityIndex++)
        {
          FTransform& Transform = TransformList[EntityIndex].GetMutableTransform();
          FVector& MoveTarget = SimpleMovementList[EntityIndex].InitialTarget;

          // ...Computation using fragment data
        }
    });
}
```

Inside the lambda, the `FMassExecutionContext` provides specific functions to access data based on the requirements defined in `ConfigureQueries` (See Table 3.2).

To scale processing for thousands of entities, the framework provides `ParallelForEachEntityChunk`. This method is functionally similar to the serial `ForEachEntityChunk` but leverages the Unreal Engine Task Graph to distribute the workload across all available CPU cores. For example, this means that while one thread is calculating movement for a batch of entities in Chunk A, another thread can simultaneously process Chunk B (chunk-level parallelization).

However, shifting to parallel execution introduces strict thread-safety requirements: the logic inside the lambda must be re-entrant and avoid writing to shared variables or class members without synchronization. Furthermore, any structural changes, such as adding fragments or destroying entities, cannot be performed immediately. Instead, these operations must be queued through the `FMassExecutionContext` command buffer, which Mass flushes safely on the main

Table 3.2: Mass Execution Context Access Functions

Requirement	Function	Description
ReadOnly Fragment	`GetFragmentView`	Returns a `TConstArrayView` containing read-only fragment data.
ReadWrite Fragment	`GetMutableFragmentView`	Returns a `TArrayView` for modifying fragment data.
ReadOnly Shared	`GetConstSharedFragment`	Returns a constant reference to a shared fragment.
ReadWrite Shared	`GetMutableSharedFragment`	Returns a mutable reference to a shared fragment.
ReadOnly Subsystem	`GetSubsystemChecked`	Returns a constant reference to a world subsystem.
ReadWrite Subsystem	`GetMutableSubsystemChecked`	Returns a mutable reference to a world subsystem.

thread once the parallel tasks conclude. This approach ensures maximum hardware utilization while maintaining the integrity of the underlying memory archetypes.

Observer Processor

While standard processors run every frame, Observer Processors (inheriting from `UMassObserverProcessor`) are event-driven. They respond to structural changes in the entity's lifecycle.

Observers are tied to specific operations on fragments or tags:

- **OnAdded**: Triggered when a component is added to an entity.
- **OnRemoved**: Triggered when a component is removed.

This is critical for initialization and cleanup logic. For example, an observer might listen for the addition of a `FTransformFragment` to register the entity with a spatial grid, or listen for the removal of a `FHealthFragment` to trigger a death animation or VFX. By using observers, the system avoids polling and instead reacts only when the change occurs.

3.2 Additional Mass Plugins

While MassEntity provides the foundational memory architecture and processing pipeline, Unreal Engine extends its capabilities through a suite of specialized plugins. These modules, developed and tested during the creation of the City Sample technical demo, provide the high-level logic required to transform raw entities into complex, autonomous agents. As of Unreal Engine 5.6, these systems remain marked as Experimental, signaling that while they are architecturally robust, their APIs continue to evolve to meet the needs of large-scale open-world simulations, constituing a production risk.

3.2.1 MassGameplay

The MassGameplay plugin acts as the primary bridge between the data-oriented ECS and the traditional Actor-based environment of Unreal Engine. It provides the essential gateway for world interaction, representation, and lifecycle management.

- **Mass Representation Subsystem:** responsible for the visual lifecycle of entities. It utilizes an intelligent Actor Pooling system to switch between high-fidelity Actors for nearby agents and lightweight Instanced Static Meshes (ISM) for distant crowds, maintaining visual density without the overhead of thousands of ticking Actors.
- **Mass LOD Subsystem:** a high-performance spatial query system that calculates the Level of Detail (High, Medium, Low, Off) for every entity. This LOD state is then utilized by other subsystems (Representation, Replication, and Movement) to scale computational costs based on viewer proximity.
- **Mass Replication Subsystem:** implements a specialized one-way (Server-to-Client) replication pipeline. By operating directly on memory fragments rather than Actor properties, it enables the synchronization of massive entity populations across the network.
- **Mass Signals and StateTree:** these subsystem integrate hierarchical state machines with the ECS. Mass Signals provides a lightweight notification system to wake up entities for processing, while the StateTree Subsystem allows agents to execute complex, multi-state behaviors driven by a unified, signal-based logic flow.
- **Mass SmartObject Subsystem:** links entities to the SmartObject database. This allows agents to query, reserve, and interact with environmental objects (e.g., chairs, doors, or vending machines) via spatial partitioning, enabling rich environmental interactions at scale.

3.2.2 MassAI

The MassAI plugin provides the structural framework for agent intelligence by defining standardized data interfaces and execution pipelines. Architecturally, it does not implement specific behaviors. Instead, it provides the abstractions and execution schemas required to bridge the Mass ECS with decision-making systems like StateTree.

- **MassAIBehavior:** defines the foundational slots for AI logic. It provides the fragments and processors necessary to manage an entity's internal thought state, acting as the host for StateTree execution and goal-oriented transitions.
- **MassAIDebug:** offers a diagnostic layer that peeks into the abstract containers. It provides visualization for internal AI states and diagnostic tools that are essential for troubleshooting the non-deterministic behaviors of large-scale agent populations.
- **MassAIReplication:** specialized for the AI domain, this module ensures that the results of the AI's decision-making are synchronized across the network. It prioritizes the replication of high-level state changes, utilizing the LOD system to ensure that bandwidth is only spent on AI logic that is currently relevant to the client.

3.2.3 MassCrowd

While the previous plugins provide the generic tools for AI and gameplay, MassCrowd is the specialized layer designed for the high-density simulation of pedestrian populations. It was the primary vehicle for the crowd logic seen in the City Sample, focusing on the transition from individual agent navigation to collective flow. The plugin's high-level responsibilities include:

- **Crowd Visualization and Animation:** it provides specialized processors that allow the crowd to have varied, vertex-animated appearances that are synchronized with their movement speed, enabling thousands of unique animations without the skeletal mesh overhead.
- **Navigation Specialization (ZoneGraph Integration):** MassCrowd implements the specific logic required for agents to follow ZoneGraphs, the lightweight, lane-based navigation paths that replace traditional NavMesh. It handles lane-switching and intersection logic, allowing entities to respect the rules of a structured urban environment.
- **Proximity and Avoidance Tuning:** building upon the base avoidance systems, this plugin includes specific tuning for pedestrian dynamics. It manages how agents wait at crosswalks or navigate around obstacles, ensuring that high-density groups do not result in clumping or collision-based artifacts.

Chapter 4

Large-Scale Entity Representation

The true power of a Data-Oriented simulation is often measured by its visual density, the ability to render thousands of individual agents without compromising the frame rate or visual fidelity. However, as the entity count scales up, the primary bottleneck shifts from CPU logic to GPU throughput and draw calls.

In traditional game developement, the visual presence of an object is inextricably linked to its logical container, in Unreal Engine this container is typically the `AActor`. In the Mass Framework this link is severed. Mass Representation serves as the managing layer that translates abstract entity data into high-performance visual primitives, primarily batching thousands of individual entity transforms into a single Instanced Static Mesh draw call.

This chapter explores this architectural shift by treating representation as a dynamic state rather than a static component property. We begin by diagnosing the inherent limitations of the Actor-centric model, specifically analyzing how `UObject` overhead and individual draw calls create a performance ceiling that traditional architectures cannot breach.

Once these bottlenecks are established, the discourse moves to the Mass LOD system, which serves as the simulation's brain for visibility and performance throttling, orchestrating agent fidelity based on spatial relevance. This leads into a technical examination of standard representation strategies, where we analyze the mechanics of Instanced Static Meshes (ISM) and the AnimToTexture pipeline, the dual frameworks that enable high-fidelity crowd visualization at scale. Finally, we explore alternative and non-standard approaches, such as Niagara-based representation and hybrid solutions.

4.1 Actor-based Representation Limitations

In the standard Unreal Engine workflow, the `AActor` serves as a monolithic container that inextricably links an object's logical state with its visual presence. While this is ideal for individual player characters, it introduces some critical bottlenecks when scaling to thousands of agents.

The `UObject` system comes with a computational overhead: each actor requires dedicated memory for reflection metadata, property tracking and integration with the garbage collection system. Simply maintaining the memory footprint of thousands empty actors can saturate the system resources even before simulation logic is processed.

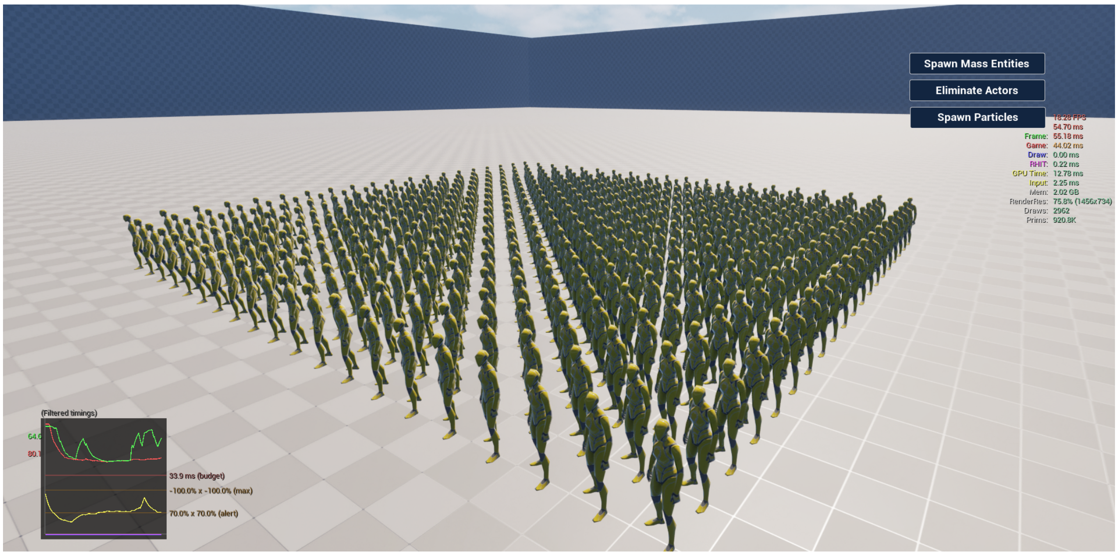

Figure 4.1: CPU performance tanking when a grid of actors is instanced in an empty scene. The Game Thread took 44ms to process this frame, hence lowering the FPS.

This overhead extends into the rendering pipeline, creating a draw call ceiling. Because each Actor's `UStaticMeshComponent` typically generates its own unique render commands, the CPU's command buffer becomes a critical bottleneck. The CPU spends so much time preparing and submitting tens of thousands of individual instructions that it cannot keep pace with the GPU, leaving the graphics hardware idle while the CPU-side rendering thread is saturated.

Finally, the architecture suffers from sequential scene updates driven by the hierarchical `USceneComponent` system. Every frame, the engine must recursively calculate world-space transforms, update bounding volumes, and perform individual visibility culling checks for every agent on the Game Thread. When the number of entities scales, this transform overhead alone can exceed the entire 16.6ms per frame budget, making real-time performance impossible within the traditional Actor-centric paradigm.

4.1.1 The Mass Solution

Mass Representation addresses the limitations of the Actor model by severing the link between an entity's logical existence and its visual manifestation. In this paradigm, the entity remains a lean data construct, while its physical presence in the world is managed by a dedicated translation layer. This architectural shift replaces the static one-to-one relationship of Actors with a dynamic visual proxy system, where the method of rendering is merely a transient state determined by the entity's current relevance to the observer.

The efficiency of this solution is rooted in the way the Mass Representation Processor interacts with the entity's existing spatial data. Instead of each agent being responsible for its own rendering, the processor iterates over the `FTransformFragment` data in a high-performance, cache-aligned stream. This allows the system to gather the transforms of thousands of entities simultaneously and push them into a centralized rendering backend.

The primary tool for this large-scale visualization is the Instanced Static Mesh (ISM), or more specifically, the Hierarchical Instanced Static Mesh (HISM). By batching these gathered transforms into a single component, Mass collapses what would traditionally be thousands of individual draw calls into a handful of GPU-optimized operations. This is critical because every draw call represents a significant communication overhead from the CPU to the GPU. In a non-instanced scenario 30000 entities, each with a mesh and unique material attributes, would generate upwards of 60000 draw calls, overwhelming the CPU's command buffer and leading to a complete collapse in frame rate.

Through the Mass Representation Subsystem, these entities are not merely rendered, but managed through a transactional phase. Instead of individual updates, the system pushes contiguous arrays of transforms and Per-Instance Custom Data (PICD), such as animation indices or color variations, to the GPU buffers in a single, streamlined operation. This system does not apply a uniform representation to all entities; instead, it utilizes a multi-tiered Level of Detail (LOD) logic to determine whether an entity should be rendered as a high-fidelity Actor, a high-performance HISM instance, or culled entirely based on its spatial relevance to the viewer.

4.2 Mass Level Of Detail

The Mass LOD system serves as the central decision-making engine for entity relevance. Unlike traditional LOD systems that only swap mesh resolutions, Mass uses LOD to drive a multi-system load balancing strategy. By categorizing entities into four distinct states (High, Medium, Low, and Off) the framework can dynamically throttle everything from rendering fidelity to the frequency of C++ logic updates.

4.2.1 Mass LOD Overview

The LOD Subsystem does not act alone, it serves three primary clients that use its spatial calculations to optimize different hardware bottlenecks:

- **Mass Visualization LOD**: manages the visual proxy swapping discussed in the previous section. It determines if an entity should be an Actor, an ISM, or culled entirely based on distance and camera frustum visibility
- **Mass Simulation LOD**: implements time throttling and variable ticking. It allows the CPU to skip update frames for distant entities. For example, we could be updating a distant agent's logic 5 times per second instead of 60.
- **Mass Replication LOD**: optimizes network bandwidth by reducing the frequency of data synchronization for entities far from the player

4.2.2 Mass LOD Subsystem

The `UMassLODSubsystem` acts as the global state provider for spatial relevance, serving as the synchronization point between the Game Thread's camera data and the Mass ECS simulation. In a high-density simulation, having tens of thousands of entities independently query the `APlayerController` for camera coordinates would create a relevant CPU bottleneck. The subsystem solves this by centralizing viewer management, ensuring that the heavy lifting of gathering engine-side world state is performed only once per frame.

Technically, the subsystem is hooked into the `PrePhysics` phase of the `UMassSimulationSubsystem`. At the start of this phase, it iterates through all active controllers, camera components, and World Partition streaming sources to populate a internal temporary array of `FViewerInfo` structures. Each entry in this array serves as a lightweight snapshot of an observer's perspective, encapsulating the following properties:

- **Location and Rotation**: he world-space origin and orientation of the viewer

- **Field of View (FOV) and Aspect Ratio**: critical parameters used to derive the frustum planes for visibility culling

This centralized array acts as the source of truth for all downstream LOD clients. Rather than entities pulling data from the world, the `UMassLODCollectorProcessor` pushes the subsystem's cached viewer data into the entity fragments. To determine how an entity archetype should interact with this viewer data, Mass provides two primary collector traits that define the policy for relevance calculation:

- `UMassDistanceLODCollectorTrait`: this trait is the more computationally lean of the two. It adds the `FMassCollectDistanceLODViewerInfoTag`, triggering the `UMassLODDistanceCollectorProcessor`. This processor calculates relevance based purely on radial proximity to the nearest viewer, ignoring the viewer's orientation. This is ideal for systemic logic, such as background AI, that must remain active even if the agent is behind the camera.

- `UMassLODCollectorTrait`: the standard collector trait adds the `FMassCollectLODViewerInfoTag`, triggering the `UMassLODCollectorProcessor`. This version utilizes the full frustum data provided by the subsystem to perform a dot-product based visibility test. It is the primary choice for visualization, allowing the system to demote entities to an Off state the moment they exit the camera's view.

Rather than entities pulling data from the world, these processors push the subsystem's cached viewer data into the entity fragments. This is facilitated by the `TMassLODCollector` processor, which performs high-speed comparisons between the global `FViewerInfo` array and the local `FTransformFragment` of each entity.

The output of this process is the population of the `FMassViewerInfoFragment`. By storing the `ClosestViewerDistanceSq` and `ClosestDistanceToFrustum` directly on the entity, Mass allows other systems, such as the Simulation and Representation processors, to perform constant-time checks on an entity's relevance. This architecture effectively decouples the complexity of the world's viewing conditions from the per-entity logic, ensuring that even with a large number of agents, the overhead of determining visibility remains a linear, cache-friendly operation.

4.3 Mass Representation

The `UMassRepresentationProcessor` is the execution engine that synchronizes an entity's logical state with its visual manifestation. It operates by evaluating the results stored in the `FMassViewerInfoFragment` and the current LOD tags to determine the appropriate `EMassRepresentationType`. This enum acts as a state machine for the entity's visual proxy, supporting four primary modes:

- **HighResSpawnedActor**: the entity is represented by a full, high-fidelity `AActor`. This is reserved for the highest LOD (typically `EMassLOD::High`), enabling complex skeletal mesh animations, physics interactions, and full component functionality.
- **LowResSpawnedActor**: a lightweight version of the Actor representation. This is often used for entities that are visible but not in the immediate foreground. It may swap the complex skeletal mesh for a simpler one or disable expensive components, such as complex AI or ticking components, while maintaining the `AActor` wrapper for basic interaction or sound.
- **StaticMeshInstance**: the entity is rendered via the HISM (Hierarchical Instanced Static Mesh) system. This is the ideal representation for performance, allowing thousands of agents to be drawn in a few GPU batches. In this state, the entity no longer exists as an `AActor` in the world; it is merely a transform and a set of custom data in a GPU buffer.
- **None**: The entity is completely culled. The mesh is removed from the render pipeline entirely, though the underlying Mass simulation continues to run.

The transition between these states is managed through the `FMassRepresentationFragment`, which tracks both the `CurrentRepresentation` and `PrevRepresentation`. This allows the system to handle the "setup and teardown" of visual assets, such as calling the `UMassActorSpawnerSubsystem` to spawn an Actor or clearing an index from a HISM, only when a state change occurs, preventing expensive per-frame logic.

4.3.1 Vertex Animated ISM

For large crowds of entities, traditional skeletal mesh skinning (calculating bone offsets on the CPU) is a primary bottleneck. A way to bypass this is found by leveraging AnimToTexture (Vertex Animation Texture).

In this workflow, animation data is baked into a 2D HDR texture where rows represent bones and columns represent time frames. During the `StaticMeshInstance` representation phase, the HISM vertex shader samples this texture to offset vertices directly on the GPU. This transforms the computational cost of an animated character into that of a static mesh with a specialized material. To ensure the crowd does not appear synchronized, it is possible to push Per-Instance Custom Data (PICD) via Mass to the HISM. In the standard AnimToTexture shader at scale, the two critical floats are typically:

- **Animation State / Index**: This tells the shader which row of the baked texture to read, allowing to select the animation among the baked ones. This is often a quantized float derived from the `FMassCrowdAnimationFragment`.
- **Normalized Animation Phase**: Instead of a global time offset, Mass frequently calculates and pushes the actual playback position (as a value going from 0.0 to 1.0) of the animation loop. This is calculated on the CPU by taking the entity's internal clock, adding a random seed, and modulo-ing it by the animation duration.

The practical application of this vertex-driven animation pipeline is most notably demonstrated in Unreal Engine's City Sample project, where the `UMassCrowdAnimationProcessor` serves as a specialized extension of the core Mass framework to orchestrate the movement of thousands of simulated pedestrians.

4.4 Niagara Representation